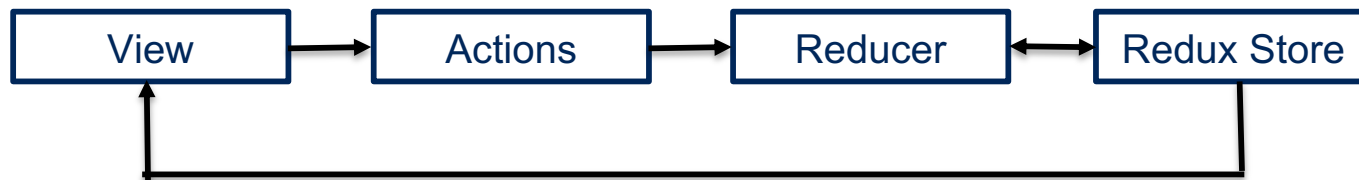


Homework



Homework

- In this week's homework, you will learn to create a simple react-redux app.
- This app will be created using codesandbox. Head on to the website and logon.
- Then create a new sandbox react app. Name this app: **travel-app**.
- To make things simple, all components, including redux will be created in App.js.
- The app will not require any states data in the main component as you are using redux as state/data management. To do a quick recap on how redux work:





Homework

- Open App.js. Remove everything in that file.
- Add the following in the dependencies:
 - redux
 - react-redux
 - bootstrap
- Add the following imports in App.js:

```
import React from "react";  
import { createStore } from "redux";  
import { Provider, connect } from "react-redux";
```

- And this in index.js:

```
import "bootstrap/dist/css/bootstrap.min.css";
```



Homework

- Return to App.js and add this set of data below the imports:

```
/* Data */  
const COUNTRIES = [  
  {  
    country: "Italy",  
    id: 0,  
    visited: "Yes",  
    year: "2012"  
  },  
  {  
    country: "Japan",  
    id: 1,  
    visited: "No",  
    year: "2023"  
  },  
  {  
    country: "Chile",  
    id: 2,  
    visited: "No",  
    year: "2025"  
  }  
];
```



Homework

- Below the data, let's create a main component that will assemble the main child components using jsx. No state data is needed here.

```
class Main extends React.Component {  
  render() {  
    return (  
      <div className="p-3">  
  
        </div>  
    );  
  }  
}
```



Homework

- Within this component, add an app title and the 1st child component to call along with props to pass to it:

```
return (  
  <div className="p-3">  
    <h1 className="text-danger">My Travel Plans</h1>  
    <hr />  
    <AddCountry addCountry={this.props.addCountry} />  
    <hr />  
  </div>  
)
```



Homework

- Add the 2nd child component to be called along with props pass to it as well. Above it, create 3 headings for the data output later:

```
<AddCountry addCountry={this.props.addCountry} />
<hr />
<div className="mx-2">
  <th className="w-25 text-center">Country</th>
  <th className="w-25 text-center">Visited</th>
  <th className="w-25 text-center">Year</th>
  <CountriesList countries={this.props.countries} />
</div>
```



Homework

- Next – let's create the two child components that will be called in the main component. Start with the 2nd child component first. Add this below the main component:

```
const CountriesList = (props) => {  
  return (  
    <ul className="list-group">  
      {props.countries.map((country) => (  
        <li key={country.id} className="list-group-item">  
          <Country country={country} />  
        </li>  
      ))}  
    </ul>  
  );  
}
```




Homework

- And now the 1st child component. Add this below the previous component:

```
class AddCountry extends React.Component {  
  constructor(props) {  
    super(props);  
  
    this.inputCountry = this.inputCountry;  
    this.inputVisited = this.inputVisited;  
    this.inputYear = this.inputYear;  
  
  }  
  
  render() {  
    return (  
        
    )  
  }  
}
```



Homework

- Within return(), add a form. Start with 3 labels and a button:

```
return (  
  <form onSubmit={this.handleSubmit} className="mx-3">  
    <label for="country">Country </label>  
  
    <label for="visited"> Visited? </label>  
  
    <label for="year"> Year Visited/To Visit </label>  
  
    <button type="submit"> Add Country</button>  
  </form>  
)
```



Homework

- Below each label, add the following input fields:

```
<label for="country">Country </label>
<input
  id="country"
  type="text"
  ref={this.setTextInputRef}
  className="d-block mb-2"
/>
```

```
<label for="visited"> Visited? </label>
<input
  id="visited"
  type="text"
  ref={this.setTextInputRef}
  className="d-block mb-2"
/>
```

```
<label for="year"> Year Visited/To Visit </label>
<input
  id="year"
  type="text"
  ref={this.setTextInputRef}
  className="d-block mb-3"
/>
```

What is Ref?

<https://reactjs.org/docs/refs-and-the-dom.html#text>



Homework

- With the form setup, you will need to create the handler next to handle the form submission which will grab the input values from the fields and set them as redux action. Add this above the render() but below the constructor:

```
handleSubmit = (e) => {  
  e.preventDefault();  
  this.props.addCountry(  
    this.inputCountry.value,  
    this.inputVisited.value,  
    this.inputYear.value  
  );  
  e.target.reset();  
}
```



Homework

- Next create a Ref function to get the input data to update the local state and send back to the view for any changes. Using the Ref attribute in the fields and a function help get updates without having to re-render props everytime a change happens.

```
this.inputCountry = this.inputCountry;
this.inputVisited = this.inputVisited;
this.inputYear = this.inputYear;

this.setTextInputRef = (inputElement) => {
  console.log(inputElement.id);
  switch (inputElement.id) {
    case "country":
      this.inputCountry = inputElement;
      break;
    case "visited":
      this.inputVisited = inputElement;
      break;
    case "year":
      this.inputYear = inputElement;
      break;
  }
}
```



Homework

- The next component displays all the data that are in the state. This component is called in the CountriesList component:

```
const Country = (props) => {  
  return (  
    <tr className="country">  
      <td className="w-25 px-2 text-center">  
        {props.country.country}  
      </td>  
      <td className="w-25 px-3 text-center">  
        {props.country.visited}  
      </td>  
      <td className="w-25 px-3 text-center">  
        {props.country.year}  
      </td>  
    </tr>  
  );  
}
```



Homework

- Next – redux. The 1st redux component is action. Again, action is initiated from the view (form) and the handler in turn send the information to this redux action creator function to create the action. Each action comes with two pieces of information: 1. the type of action 2. the data or payload. Add this below the previous class component:

```
/* ACTION TYPE */
const ADD_COUNTRY = "ADD_COUNTRY";

/* ACTION */
const addCountry = (country, visited, year) => ({
  type: ADD_COUNTRY,
  payload: {
    country: country,
    visited: visited,
    year: year
  }
});
```



Homework

- The action is now dispatch to another redux function called reducer. The job of the reducer is to do two things: 1. update the data in the state 2. sends the update back to the view. This is done by matching the right action type with the incoming action type to perform. It then takes the new data and perform state updating. Add this reducer after the redux action:

```
/* REDUCER */
const Reducer = (state = COUNTRIES, action) => {
  switch (action.type) {
    case ADD_COUNTRY:
      let country = action.payload;
      country.id = state.length;
      //instructor.color = '#000';
      return state.concat(country);
    default:
      return state;
  }
};
```




Homework

- Next - the store itself. To store/update data in the redux store, it needs to be created first. The two functions deals with data in the store: 1. `mapStateToProps()` 2. `mapDispatchToProps()`. As you guess it – `mapStateToProps` deals with state and returns as `stateProps`, whereas `mapDispatchToProps` deals with dispatch and returns `dispatchProps` to the connected components by passing the action creator function. Add this below the reducer.

```
/* STORE & CONNECTIONS TO STORE */
const store = createStore(Reducer, COUNTRIES);

const mapStateToProps = (state) => {
  return {
    countries: state
  };
};

const mapDispatchToProps = (dispatch) => ({
  addCountry: (country, visited, year) =>
    dispatch(addCountry(country, visited, year))
});
```



Homework

- With the redux action creator, redux reducer and redux store set up, it's time to connect the main component to the store. Why the main component? Because this component is the central hub that brings all other components and redux together. To connect, pass both `mapStateToProps` and `mapDispatchToProps` to the store connection. But this is stored in a variable first before it actually gets connected (not mandatory, only for this case). This can be done at the very end of the codes.
- Add this after the store set up:

```
const AppExport = connect(  
  mapStateToProps, mapDispatchToProps)(Main);
```

More information about redux connect, `mapStateToProps` and `mapDispatchToProps`:
<https://react-redux.js.org/api/connect>



Homework

- And to finally get connected to the store while also let all nested components access the same redux state data, the provider component allows just that. For this, wrap the connection with this provider component and pass to it our store that was set up earlier.
- To make things easier and cleaner, put this in the App component so it can be exported and push to the index.

```
const App = () => {  
  return (  
    <Provider store={store}>  
      <AppExport />  
    </Provider>  
  );  
}
```

And finally export the App as the default component. Add this at bottom of the page:

```
export default App;
```



Homework

- Testing the app:

My Travel Plans

Country

Visited?

Year Visited/To Visit

Country	Visited	Year
Italy	Yes	2012
Japan	No	2023
Chile	No	2025



My Travel Plans

Country

Visited?

Year Visited/To Visit

Country	Visited	Year
Italy	Yes	2012
Japan	No	2023
Chile	No	2025
Canada	Yes	2009

Homework

- **DUE:**

On or before this Sunday, 10.30PM PT

- **Submission:**

Post Codesandbox link on GAP Week 6 Day 1 Homework



Connect with Us (WEB503/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Christopher Paul

christopherpaul@westcliff.edu

Teaching
Assistant

Ilya Valasov

ilyavalasov@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

End of Presentation